4-bit Unsigned Array Multiplier from RTL to GDSII on SkyWater SKY130 with Open-Source Tools

Sai Poojith
Roll No. 2025122010

Aneesh
Roll No. 2025122002

***Abstract*—We designed a 4-bit × 4-bit unsigned array multiplier in Verilog with registered inputs and outputs (16 flip-flops, 2-cycle latency), verified it exhaustively, synthesised it to the SKY130 `sky130_fd_sc_hd` library with Yosys, analysed its timing with OpenSTA and took it to a GDSII layout with LibreLane. RTL and gate-level simulation both pass all 256 input combinations. Synthesis gives 73 cells (805.773 μm^2) and STA gives $T_{\min} = 2.596$ ns ($F_{\max} = 385.2$ MHz) at the typical corner. LibreLane needed 8 runs and closed timing at a 7.0 ns clock across all its default corners, with 0 DRC and 0 LVS errors. Post-route $F_{\max}$ is 150.70 MHz at the worst (slow) corner and 302.91 MHz at the typical corner.**

I. Part A — RTL Design and Verification

A. Design

`rtl/full_adder.v` is a 1-bit full adder written only with `assign` statements: $sum = a \oplus b \oplus c_{in}$ and $c_{out} = ab + bc_{in} + ac_{in}$. `rtl/mult_array.v` registers `a` and `b` into `a_reg`/`b_reg` (8 flip-flops), forms the 16 partial-product bits $pp_i[j] = a[j] \wedge b[i]$, and adds them with three rows of four `full_adder` instances (`fa_r1_0` … `fa_r3_3`, 12 in total) wired exactly as in the connection table of Section 2 of the assignment. The 8-bit sum is registered into `p` (8 flip-flops). All 16 flip-flops are cleared by the asynchronous active-low reset `rst_n`. No `*` operator is used in the design. Fig. 1 shows the block diagram.

B. Testbench

`tb/tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles and applies all 256 combinations of a and b. Each input is applied on a falling edge; the testbench waits two rising edges (the 2-cycle latency) and compares `p` with `a*b` at the next falling edge. It prints a line for every mismatch, one summary line at the end, and dumps `mult.vcd`.

C. Results

RTL simulation:

```
PASS: 256 vectors checked, 0 errors
```

Error test (task 6), with `sum` changed to `a ^ b`:

```
MISMATCH: a=3 b=3 expected=9 got=5
...
FAIL: 256 vectors checked, 95 errors
```

After undoing the change the RTL simulation prints `PASS: 256 vectors checked, 0 errors` again.

D. Hand-worked example

Sai Poojith's roll number 2025122010 ends in 0, so $A = 0 + 5 = 5$ and $B = 7$.

```
        0 1 0 1  A = 5
      x 0 1 1 1  B = 7
---------------
        0 1 0 1  pp0 = A AND b0 (b0=1)
      0 1 0 1    pp1 = A AND b1 (b1=1), shifted 1
    0 1 0 1      pp2 = A AND b2 (b2=1), shifted 2
  0 0 0 0        pp3 = A AND b3 (b3=0), shifted 3
---------------
0 0 1 0 0 0 1 1  P = 35
```

Fig. 2 shows the same multiplication in GTKWave. The inputs $a = 5$, $b = 7$ are applied at 1760 ns, captured into the input registers at clock edge 1 (1765 ns), and the product $p = 35$ appears after clock edge 2 (1775 ns): a latency of 2 clock cycles.

II. Part B — Logic Synthesis

`synth/synth.tcl` runs the seven Yosys steps from the assignment (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat`, `write_verilog`) with the `tt_025C_1v80` Liberty file. Table I is taken from `synth/mult_array_stat.rpt`.

TABLE I
Part B results (Yosys)

Metric	Value
Total number of cells	73
Number of flip-flops	16 (`dfrtp_1`)
Total cell area (μm^2)	805.773
Flip-flop area (μm^2)	$16 \times 25.024 = 400.384$
Flip-flop area (% of total)	49.69 %
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (11), then a four-way tie at 4: `a21oi_1`, `and2_0`, `nand4_1`, `xnor2_1`
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

The gate-level netlist was simulated with the assignment's command (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and the same testbench, and passed. Our first gate-level run printed `FAIL: 256 vectors checked, 239 errors`. Every wrong value was the product of the previous vector. The testbench was sampling `p` 1 ns after the clock edge, which is exactly when the unit-delay flip-flop models update their

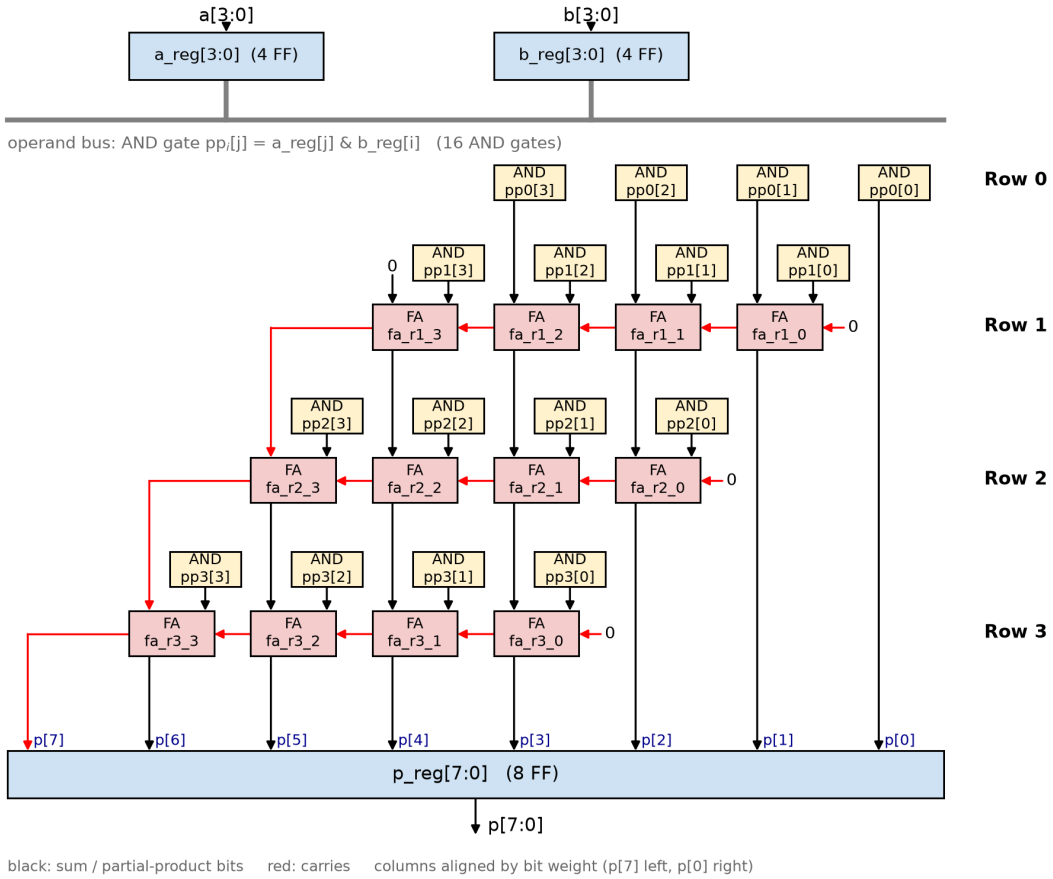


Fig. 1. Block diagram of `mult_array`: input registers, the 16 AND gates, the 12 full adders (labelled with their RTL instance names) and the 8-bit output register. Columns are aligned by bit weight; black arrows are sum and partial-product bits, red arrows are carries.

outputs. We moved the comparison to the falling edge and reran all Part A and Part B simulations.

III. Part C — Static Timing Analysis

`sta/sta.tcl` runs the seven OpenSTA steps from the assignment at the typical corner. At $T = 10$ ns the worst setup slack is 7.404 ns, so

$$T_{\min} = 10 - 7.404 = 2.596 \text{ ns},$$
$$F_{\max} = 1000/2.596 = 385.2 \text{ MHz}.$$

Rerunning with $T = 2.60$ ns gives a slack of 0.004 ns (printed as `worst slack max 0.00`), which confirms $T_{\min}$.

The path starts at `b_reg[0]`, goes through the partial-product AND gate for $pp_0[0]$, then through the carry logic of the adder rows, and ends at the sum output of `fa_r3_3`, which is `p[6]`. In the netlist, `_083_` really is $a[0] \wedge b[0]$ (it also drives `p_reg[0]`). In the RTL $pp_0[0]$ only drives $p[0]$, but row 1's first carry $c_1[1] = pp_1[0]\,pp_0[1] = (a_0 b_1)(a_1 b_0)$ equals $pp_0[0]\,pp_1[1]$, so ABC reused this AND gate in the carry logic (it feeds `_054_`–`_057_` together with NAND(a_1, b_1) and NAND(a_2, b_0)). Of the 8 logic gates, 1 is a partial-product AND gate and 7 (rows 3–9 in Table III) implement full-adder sum and carry logic. The estimate in Section 2 is about $2N =$ 8 full-adder delays. Because the design was flattened, ABC was free to merge adjacent full-adder stages into larger single cells such as `and4_1`, `o2111a_1` and `xnor3_1`, so 7 cells cover the work of about 8 full-adder delays.

TABLE II
Part C results (OpenSTA, tt_025C_1v80)

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.404
$T_{\min}$ (ns)	2.596 (rerun at 2.60)
$F_{\max}$ (MHz)	385.2
Slack at $T = T_{\min}$ (ns)	0.004
Critical path start point	`_118_` = `b_reg[0]` (`dfrtp_1`)
Critical path end point	`_112_` = `p_reg[6]` (`dfrtp_1`, D = `fa_r3_3.sum`)
Number of cells on the critical path	10 (launch FF + 8 gates + capture FF)
Latency (ns) $= 2 \times T_{\min}$	5.192
Throughput (M results/s)	385.2

IV. Part D — Physical Design

`pnr/config.json` contains exactly the six settings from the assignment: `DESIGN_NAME`, `VERILOG_FILES`, `CLOCK_PORT`, `CLOCK_PERIOD`, `FP_SIZING` =

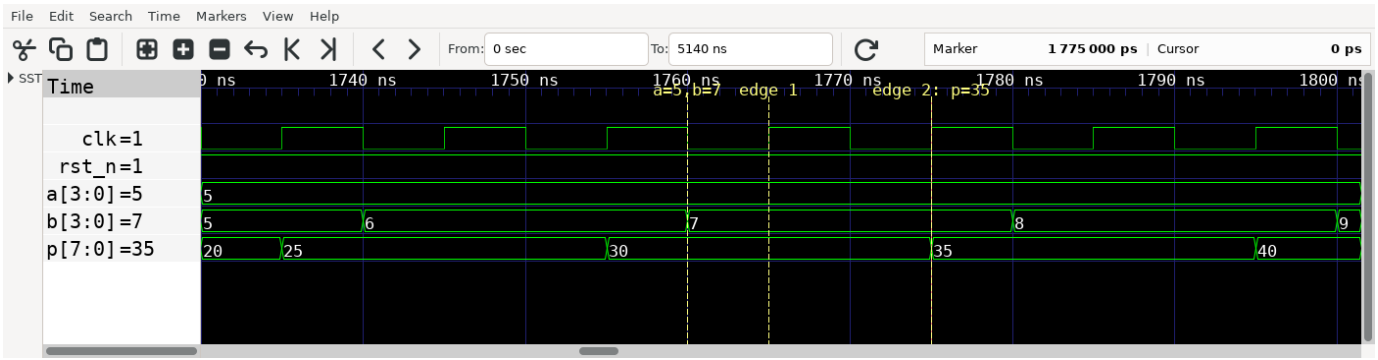


Fig. 2. GTKWave view of the 5×7 example (1730–1800 ns). Marker *a=5,b=7*: inputs applied (1760 ns); *edge 1*: inputs captured (1765 ns); *edge 2: p=35*: product registered (1775 ns).

TABLE III
CELLS ON THE CRITICAL PATH, IN ORDER (OPENSTA TIMING REPORT, $T = 2.60$ NS)

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role
1	_118_	dfrtp_1	0.395	0.395	launch flip-flop `b_reg[0]` (CLK→Q)
2	_083_	and2_0	0.212	0.607	partial product $pp_0[0] = a[0] \wedge b[0]$
3	_055_	or3_1	0.361	0.968	adder carry logic
4	_059_	and4_1	0.215	1.183	adder carry logic
5	_070_	o2111a_1	0.234	1.417	adder carry logic (also takes $a[2]$, $b[2]$)
6	_076_	o21ai_0	0.216	1.634	adder carry logic
7	_090_	xnor3_1	0.274	1.907	3-input XOR (full-adder sum)
8	_091_	maj3_1	0.375	2.282	majority (full-adder carry)
9	_092_	xor3_1	0.178	2.460	3-input XOR = `fa_r3_3.sum`
10	_112_	dfrtp_1	setup 0.135	—	capture flip-flop `p_reg[6]`

`"absolute"` and `DIE_AREA = [0, 0, 150, 150]`. The flow was run with `librelane config.json` (LibreLane v3.1.0.dev3) and its default timing corners: typical, slow and fast process corners (`tt_025C_1v80`, `ss_100C_1v60`, `ff_n40C_1v95`), each with nominal, minimum and maximum wire parasitics.

A. Timing closure

The starting period was $1.2 \times T_{\min} = 1.2 \times 2.596 = 3.115$ ns, rounded up to 3.5 ns. Every failing run failed only on setup timing in the slow corner `ss_100C_1v60`, where cells are much slower than at the typical corner used in Part C. Following the assignment, we increased `CLOCK_PERIOD` by 0.5 ns after each failure (Table IV). From 3.5 to 5.5 ns the slack improved by 0.5 ns per run, but at 6.0 and 6.5 ns it hardly improved. Synthesis was identical in all eight runs (67 cells, 738.208 μm^2), and so was the layout up to clock-tree synthesis (1224.92 μm^2 of standard cells). The runs differ only in the post-CTS timing-repair step (`ResizerTimingPostCTS`), which upsizes gates only where it predicts a setup violation from pre-route wire estimates. Up to 5.5 ns it added 101.35 μm^2 (to 1326.27 μm^2); at 6.0 ns it added only 73.83 μm^2; at 6.5 and 7.0 ns it predicted no violation and added nothing. With less upsizing the slow-corner path got slower (data arrival 5.68 ns up to 5.5 ns, 6.13 ns at 6.0 ns, 6.66 ns at 6.5 ns), so the slack stayed negative until 7.0 ns, where the same un-upsized netlist met timing.

TABLE IV
LIBRELANE RUNS (FROM EACH RUN'S FINAL METRICS)

#	Period (ns)	Result	Worst slack (ns), ss	Viol.	tt slack (ns)
1	3.5	FAIL	−2.154	15	0.587
2	4.0	FAIL	−1.654	12	1.087
3	4.5	FAIL	−1.154	12	1.587
4	5.0	FAIL	−0.654	9	2.087
5	5.5	FAIL	−0.155	3	2.587
6	6.0	FAIL	−0.107	3	2.899
7	6.5	FAIL	−0.136	3	3.199
8	7.0	PASS	+0.364	0	3.699

Worst corner in every run: `max_ss_100C_1v60`. tt = `nom_tt_025C_1v80`. All runs: 0 DRC, 0 LVS.

B. Results

Post-route $F_{\max} = 1000/(\text{CLOCK_PERIOD} - \text{setup slack})$. We give it both for the worst corner, which is what LibreLane signs off on, and for the typical corner, which is the fair comparison with Part C:

$$\text{worst (ss): } 1000/(7.0 - 0.3644) = 150.70 \text{ MHz},$$
$$\text{typical (tt): } 1000/(7.0 - 3.6987) = 302.91 \text{ MHz}.$$

The 1895 instances are 16 flip-flops and 51 logic cells from LibreLane's own synthesis, plus 1828 cells added by the flow (Table V). LibreLane runs its own Yosys script, so its logic (67 cells, 738.208 μm^2) is not identical to our Part B netlist

TABLE V
PART D RESULTS (LIBRELANE FINAL RUN, 7.0 NS CLOCK)

Metric	Value
CLOCK_PERIOD used (ns)	7.0
Number of runs needed	8 (3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0 ns)
Core area (μm^2)	17 759.5 (die area 22 500)
Cells after synthesis (Part B)	73
Cells after place and route	1895 (334 standard cells + 1561 fill/decap)
Kinds of cells added	decap 1314 (`decap_12` 905, `decap_6` 175, `decap_3` 130, `decap_8` 55, `decap_4` 49); fill 247 (`fill_1` 215, `fill_2` 32); tap 245 (`tapvpwrvgnd_1`); buffers 22: 17 I/O port buffers (`buf_2`), 3 clock-tree buffers (`clkbuf_16`), 2 fanout buffers (`clkdlybuf4s50_1`) that split the `rst_n` net (16 reset pins, above the maximum fanout of 10)
Wirelength (μm)	2362
Setup slack (ns)	worst corner (`max_ss_100C_1v60`): 0.3644; typical (`nom_tt_025C_1v80`): 3.6987
Post-route $F_{\max}$ (MHz)	worst corner: 150.70; typical corner: 302.91
$F_{\max}$ from Part C (MHz)	385.2 (typical corner, no wires)
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

(73 cells). Hold timing also passes in every corner (worst hold slack 0.141 ns). Figs. 3 and 4 show the final layout.

V. ANSWERS TO QUESTIONS

Q1. Logic cells. Synthesis produced $73 - 16 = 57$ logic cells, about twice $16+12 = 28$, because the RTL's AND gates and full adders do not map one-to-one onto library cells. The design was flattened, so ABC optimised all the logic together. It dropped constant inputs (each row's carry-in and $r_0[3]$ are 0) and merged AND gates into adder logic (`o2111a_1` takes $a[2]$ and $b[2]$ directly). It also skipped the library's multi-output `fa_1` cell; its log says it ignored 9 multi-output cells. So each full adder becomes several single-output cells, such as an XOR3 for the sum plus a MAJ3 or AND-OR cells for the carry.

Q2. Critical path. The critical path goes from input register `b_reg[0]` (`_118_`) to output register `p_reg[6]` (`_112_`, the sum of `fa_r3_3`). It has 10 cells: the launch flip-flop, 8 logic gates and the capture flip-flop. The logic part takes $2.460 - 0.395 = 2.065$ ns. One gate is the partial-product AND for $pp_0[0]$, which ABC reused in row 1's carry because $c_1[1] = pp_0[0]\,pp_1[1]$. The other 7 implement full-adder carry and sum logic, rippling through the rows to `fa_r3_3`. That is close to the $2N = 8$ full-adder estimate. It is slightly fewer cells because ABC used complex gates (`and4`, `o2111a`, `xnor3`, `maj3`) that each do the work of more than one simple full-adder stage.

Q3. Flip-flop area. The flip-flops take 400.384 of 805.773 μm^2, which is 49.69 %. Each `dfrtp_1` is 25.024 μm^2, because it contains two latches plus asynchronous reset logic. Most logic gates in our netlist are only 3.75–11.26 μm^2; just the 3 `xnor3`/`xor3` cells are close to a flip-flop in size (22.5–23.8 μm^2). The design is tiny (57 logic cells), so the 16 registers are a large share. For an N-bit multiplier the logic grows like N^2 but there are only $4N$ flip-flops, so in a bigger multiplier this percentage would drop.

Q4. Why post-route $F_{\max}$ is lower, and the added cells. Two effects lower it. The first is *multi-corner signoff*. Part C used only the typical corner, but LibreLane signs off on its slowest corner, `ss_100C_1v60`. There, on the critical path from `a_reg[0]` (`_102_`) to `p_reg[6]` (`_100_`, D = `fa_r3_3.sum`), data arrives at 6.656 ns instead of 3.247 ns at tt in the same layout, which gives the worst-corner $F_{\max}$ of 150.70 MHz. The second is *the real layout*: even at tt, $F_{\max}$ falls from 385.2 to 302.91 MHz. The tt effective period of 3.301 ns is the 2.911 ns data path (launch clock to D) plus a 0.124 ns setup time, LibreLane's 0.25 ns clock uncertainty and 0.016 ns of clock skew. The capture clock arrives 0.034 ns earlier than the launch clock (0.302 vs. 0.336 ns), less a 0.018 ns reconvergence credit. The clock-tree latency itself cancels, because it delays both flip-flops. The data path is longer than Part C's 2.460 ns for three reasons. Routed wires add RC, so the launch flip-flop's clk→Q is 0.472 ns instead of 0.395 ns. LibreLane's own synthesis gives a different 9-gate path. And at 7.0 ns the post-CTS timing repair saw no violations, so it upsized nothing. The flow also added cells: 245 tap cells tie the n-wells and substrate to VPWR/VGND to prevent latch-up, and 1314 decap cells are on-chip capacitors that steady the supply when many gates switch. 247 fill cells close the gaps in the rows so wells, implants and power rails stay continuous, and 3 `clkbuf_16` cells form the clock tree that reaches all 16 flip-flops with low skew. Finally, there are 17 `buf_2` buffers on the input and output ports and 2 fanout buffers that split `rst_n`, whose 16 reset pins exceed the maximum fanout of 10.

Q5. DRC and LVS. DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths and spacings of each layer, enclosures, via sizes, densities and so on. LVS (layout versus schematic) extracts the transistors and connections from the layout and checks that they match the netlist, so the layout is the circuit we designed. Both must pass because they catch different failures. A DRC-clean layout can still be the wrong circuit, and a layout that matches the schematic can still be impossible to manufacture reliably (shorts, opens, low yield). Our final layout has 0 DRC errors (Magic and KLayout) and 0 LVS

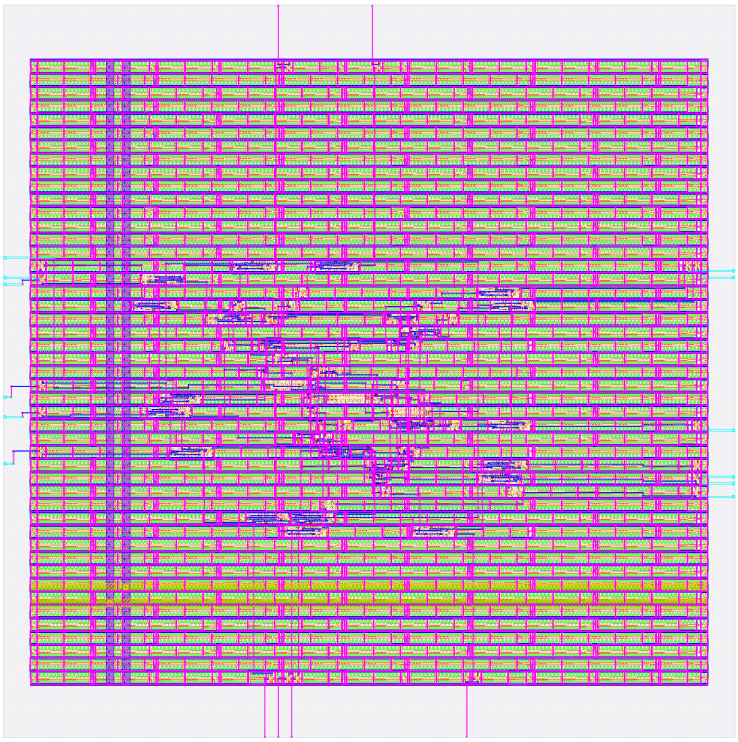


Fig. 3. Whole $150 \times 150\,\mu$m die (KLayout, `pnr/mult_array.gds`). The I/O pins sit on the die edge. The multiplier logic is clustered near the centre, and the rest of the core rows are filled with decap, fill and tap cells under the power grid.

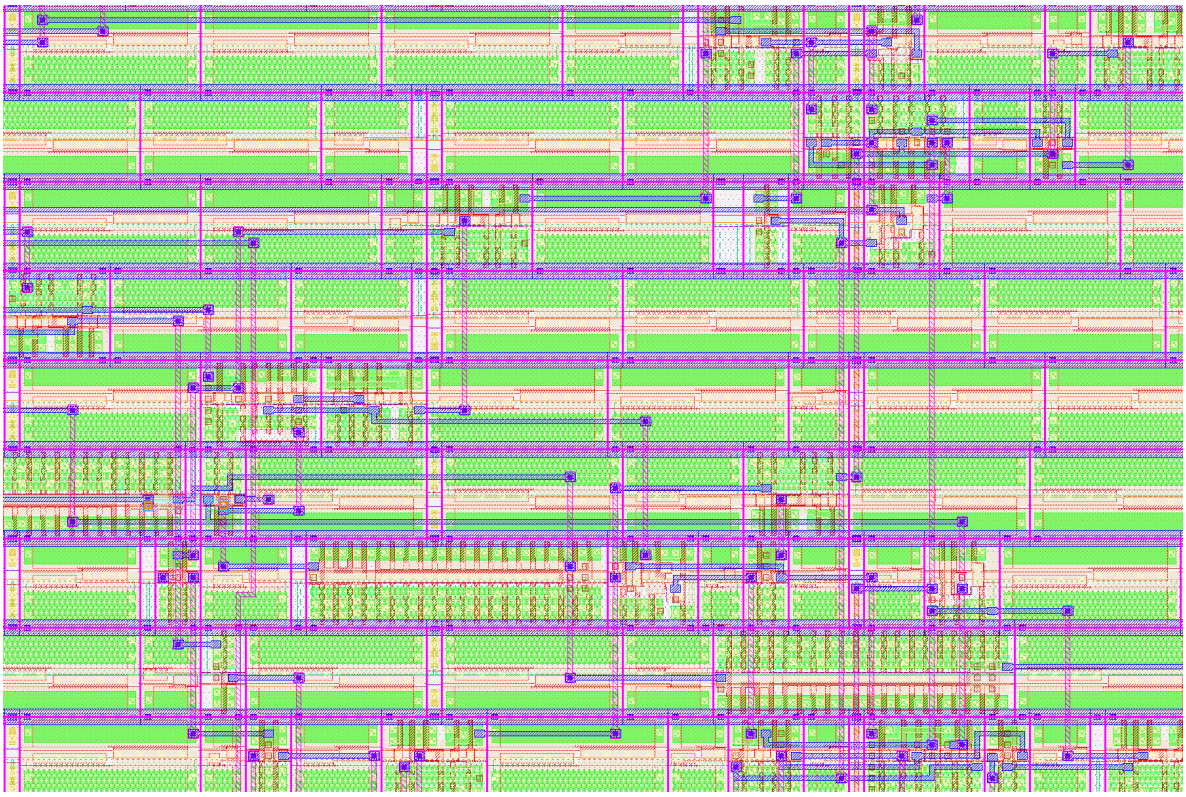


Fig. 4. Zoomed $36 \times 24\,\mu$m view at the die centre (colours from `sky130A.lyp`). Magenta outlines are the standard-cell boundaries (areaid layer); the cells sit in horizontal rows with blue metal-1 power rails along the row edges. Inside the cells, red is poly (transistor gates), green is diffusion and pale beige is li1, the local-interconnect layer that does most of the in-cell wiring ($671\,\mu\text{m}^2$ of li1 against $192\,\mu\text{m}^2$ of metal-1 in this window). Blue metal-1, a few magenta metal-2 wires and violet vias connect the cells.

errors.

Q6. Scaling to 8 bits. From Section 2, an N-bit array multiplier uses N^2 AND gates and $N(N-1)$ full adders, and its longest path is about $2N$ full-adder delays.

$$N = 4: \quad 4 \cdot 3 = 12 \text{ FAs}, \quad 2 \cdot 4 = 8 \text{ FA delays}$$
$$N = 8: \quad 8 \cdot 7 = 56 \text{ FAs}, \quad 2 \cdot 8 = 16 \text{ FA delays}$$

So the number of full adders grows by $56/12 \approx 4.7\times$ (and the AND gates from 16 to 64), but the critical path only doubles. Scaling our Part C logic delay: $T_{\min} \approx 0.395 + 2 \times 2.065 + 0.135 \approx 4.66\,\text{ns}$, so $F_{\max}$ would drop from about 385 MHz to about 215 MHz at the typical corner. Here 0.395 ns is the clk→Q delay, 2.065 ns is the logic delay and 0.135 ns is the setup time. The area grows roughly quadratically and the delay roughly linearly.